

RightFusion: Enabling QoS driven Function Fusion in Edge-Cloud FaaS

Sheshadri K R and J Lakshmi

Cloud Systems Lab

Indian Institute of Science, Bangalore 560012 INDIA

Email: sheshadrik@iisc.ac.in, jlakshmi@iisc.ac.in

Abstract—Function-as-a-Service (FaaS), a form of serverless computing, allows applications to be composed as a workflow of stateless, event-driven, short-lived functions that are deployed automatically by the service provider. FaaS has its origins in the Cloud, but recently, it has seen considerable adoption in the Edge-Computing too. FaaS exhibits significant overheads between invocations of consecutive functions and function fusion is being used to resolve these. The fused function groups are assigned the highest resource of all the functions in the fusion group and are limited by the fixed resource sizes associated with the functions; this translates to high function execution costs. In this trade-off, the function size and execution time, which is associated with input size and user-performance expectation, are not considered for fusion decisions on Heterogeneous resources. In this paper, we present “RightFusion”, a novel fusion technique that performs function-fusion on an Edge-Cloud resource pool based on application input and QoS requirements. The platform right-sizes resource instances and performs cost-efficient function fusion while meeting the QoS requirements. It also exploits heterogeneous resource cost difference to choose resources for fused functions deployment. Evaluations based on a diverse range of applications show that RightFusion reduces resource usage by above 35% at Edge and Cloud while meeting the QoS requirements compared to the PeakFusion baseline.

I. INTRODUCTION

Serverless computing as Function-as-a-Service (FaaS) offers event-driven, stateless execution of functions with execution time billing [1] that is gaining adoption in Edge computing [2], [3]. Edge devices are resource-constrained and are preferred for executing short-lived tasks closer to the point of data generation. Executing applications built as function workflows using FaaS face high overheads [4] [5] [6] [7] due to external coordination and data exchange. Function Fusion [8] has been a method to offset these overheads. While function fusion reduces the end-to-end execution times by eliminating inter-function overheads, it has an unintended consequence of high execution costs due to the peak resource assignments to the fusion group. The group is assigned the highest resource of all the functions so that every function in the fusion group can execute as per its specifications. This is termed as “Peak Fusion” in this paper. The function execution costs for the Peak Fusion group can be high since all the functions are billed based on the resource of the fusion group, irrespective of whether all functions in the group can use and benefit from the resource assignment. Alternatively, if the fusion group is assigned the minimum resource specification of all the functions in the group, some functions under-perform. Existing works [8], [9],

[10] have shown the benefits of function fusion using emerging applications that exhibit a good suitability to FaaS, such as Image inferencing and IoT (Internet of Things) workflows. However, the fusion decisions are restricted by the fixed resource specifications of the functions, which are agnostic to the varying application characteristics such as incoming input (Image Inferencing pipelines can get different image sizes, Text Inferencing may process documents of different sizes). With varying application characteristics, and a heterogeneous resource pool comprising Edge and Cloud layers, it is a challenging task to identify resource specifications to function fusion groups that results in cost-efficient executions while still meeting its desired performance.

In this paper, we consider expected application behavior through QoS attributes like performance, privacy, security etc.,. We present a novel fusion criterion called “RightFusion”, that performs QoS-aware cost-efficient function fusion across Edge and Cloud computing layers preserving user-specified performance. Further, we describe the algorithm that leverages the RightFusion criterion, by recommending non-peak resource assignments to a fusion group. The resource specification of each function in an application workflow is tailored based on incoming application characteristics and QoS requirements. In summary, this work makes the following contributions:

- Proposes a novel fusion criterion called “RightFusion” which fuses the functions by exploiting application characteristics, QoS requirements, and Heterogeneous resource pool comprising Edge and Cloud layers. The proposed algorithm leverages the RightFusion criterion to make cost-efficient executions using non-peak function fusions in a heterogeneous resource pool. The user-specified QoS requirements and constraints direct the selection of resource layer and function fusion.
- Evaluates the proposed algorithm that uses the RightFusion criterion on real-world applications to demonstrate fusion benefits. Experiments demonstrate adaptability of RightFusion in yielding above 35% cost benefits by judicious function fusion across Edge and Cloud with varying QoS requirements.

The remainder of the paper is organized as follows: Section II highlights the gaps in the existing techniques, and sets the motivation for RightFusion. Section III compares and discusses the relevant related work. Section IV describes the

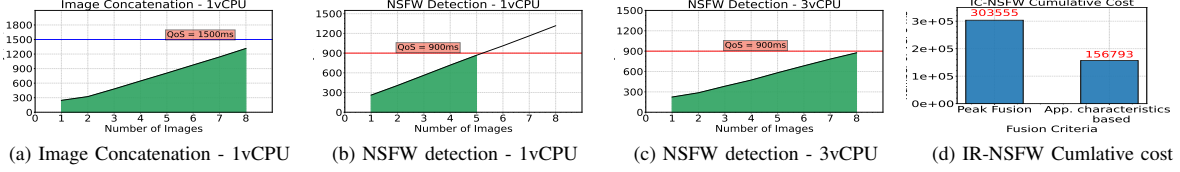


Fig. 1: IC, NSFW Execution times for different resource assignments

architecture of the proposed platform and details its major components. Section V describes the applications used for evaluation. Section VI discusses the improvements provided by the proposed platform. Finally Section VII concludes our work and sets the direction for future work.

II. GAPS AND OBSERVATIONS

This section brings out the problems associated with function fusion in the existing open-source [11] [12] and commercial offerings [13] [14], where the function executions are always performed with peak resource assignments [15]. Section II-A highlights the cost implications of peak fusion, and the challenges involved in the determining fusion groups through illustrative experiments. Section II-B summarizes the shortcomings in the existing fusion approaches.

A. Gaps

Serverless functions are deployed on an open-source serverless platform OpenLambda (OL) to highlight the shortcomings in Peak Fusion. OL was chosen for our experiments as it is designed for research prototyping [11].

1) *Peak Fusion Costs*: The costs associated with Peak Fusion are demonstrated with two illustrative functions, NSFW detection (Not Safe For Work) “*nsfw*” [16], and Image Concatenation “*ic*”, which are chosen for fusion. The *nsfw* function takes images as input and classifies them as either “Safe For Work” or “Not Safe For Work”; the *ic* function concatenates multiple images into a single image using the OpenCV library. Both functions are deployed on OpenLambda on an Intel(R) Xeon(R) Silver 4310 CPU @ 2.10GHz node with 12 physical cores; the resource construct is a Docker container. The resource assignment for *ic* is (1vCPU, 1GB RAM), and *nsfw* is configured with (3vCPU, 3GB RAM); the user-specified QoS requirement is latency and is set to 1500ms and 900ms for *ic* and *nsfw* respectively. The latency requirement reflects the threshold the function should meet the desired QoS requirement for all inputs. Function execution requests are issued to the OL controller with different number of images. The number of images are varied from 1 through 8 (to represent the variations in the number of images in real-world scenarios in web pages [17]), the image size of all the images is set to 1000KB to make it convenient for comparing execution times. The experiment is repeated for ten iterations, and average execution times are reported.

The plots in the Fig. 1a and 1c show the average execution times of *ic* and *nsfw* in milliseconds (ms) in Y-axis, for different number of images (each of 1000KB) from 1 to 8 in the

X-axis. It can be observed that *ic* and *nsfw* meet user-specified QoS requirements for all the inputs with the peak resource assignment of 1vCPU and 3vCPU, respectively. The range of input for which the functions meet the QoS requirement is highlighted in green. When these two functions are fused with peak fusion technique, the fusion group is assigned the resource size of 3vCPU, 3GB RAM. With this size, both *nsfw* and *ir* meet the user-specified QoS requirements. The execution cost for the fusion group for all the requests can be expressed as Cumulative Cost (CC). For a workload with n requests (1 to n), with a fusion group containing m functions (f_i to f_m), with the execution time of each function being t_{f_i} , the CC is expressed in the Eqn. 1.

$$CC = \sum_{req=1}^n \sum_{i=1}^m t_{f_i} * \max\{res_{f_i} : i = 1 \dots m\} \quad (1)$$

In this experiment, for a total of 80 requests, the peak fusion execution costs are highlighted in plots of Fig. 1d. Instead of sizing the *nsfw* function at 3vCPU, if it is assigned a resource size of 1vCPU as shown in plots of in Fig. 1b, it can be observed that *nsfw* function meets the QoS requirements for the input range of up to 5 images. If *nsfw* and *ir* functions are fused, and the fusion group is sized at 1vCPU and 1GB RAM, the fusion group does not violate the QoS requirements for the input range of 5 images, and for the rest of the input range (number of input images > 5), functions can be executed with peak resource assignment independently without fusion. The fusion costs of this execution are highlighted in plots of Fig. 1d as “App characteristics based fusion”. It can be observed that compared to Peak Fusion costs, the fusion based on application characteristics and QoS requirements shows 49% lower execution costs. In summary, performing function fusion with application characteristics and QoS requirements leads to lower execution costs compared to Peak Fusion which fuses functions with a peak resource size.

2) Resource Heterogeneity and Varying fusion criterion:

While the previous experiments highlight the costs of peak fusion. In this experiment, we show the variability in fusion criterion due to resource heterogeneity (Edge and Cloud layers), and varying QoS requirement. We consider two illustrative functions, Image-Resizing “*ir*”, Face-Detection “*fd*” as a choice for fusion. The *ir* function takes an image as input and outputs a resized image; the *fd* function takes an input image and detects the faces in it using HaarCascades classifier, it internally uses OpenCV module. The function *ir* is configured

with (1vCPU,1GB), *fd* is configured with (3vCPU,3GB); the user specified QoS requirement is latency and is set to 750ms and 1500ms respectively for *ir* and *fd*. Both the functions are executed with input (images) of different sizes from 50KB to 2MB. The functions are executed on Cloud device (Intel(R) Xeon(R) Silver 4310 CPU @ 2.10GHz), and Edge device (Raspberry Pi4B quad-core Cortex-A72 64-bit @ 1.5GHz).

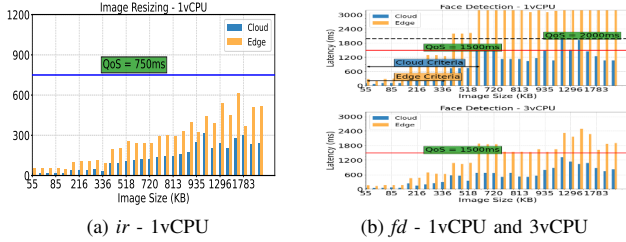


Fig. 2: IR, FD Execution times on Edge-Cloud layer

The plots in the Fig. 2a and 2b show the execution times of *ir* and *fd* on both Edge and Cloud devices; the X-axis indicates image size in KB, Y-axis indicates the execution time in milliseconds (ms). For the peak resource specification of 1vCPU for *ir*, and 3vCPU for *fd*, all the input images are executed within the given QoS requirement for Cloud device. However, a single resource specification can translate to different execution latency due to differences in compute capacity between devices. This can be observed in the plots of Fig. 2b, the execution times of Edge device is higher than that of Cloud device even though both have an identical resource assignment of (3vCPU, 3GB). The variability of execution times of functions due to resource heterogeneity brings a new challenge of determining the exact fusion criterion when fusing functions. For instance, when the *fd* function is sized at 1vCPU, the Edge device can execute *fd* until 350KB without violating QoS requirement of 1500ms (Edge Criteria label in upper plot of Fig. 2b); however the Cloud device can execute until 600KB (Cloud Criteria label in Fig. 2b).

As the performance of functions changes due to resource heterogeneity, there is also a need for the fusion criteria to dynamically adapt to it. In the considered scenario, the *ir* and *fd* functions can be fused at (1vCPU, 1GB) non-peak resource for input range in Edge devices (until 350KB input size); whereas in Cloud devices the fusion of *ir* and *fd* for the same resource assignment (1vCPU, 1GB) can satisfy the QoS requirement up to 600KB input image size. Another aspect of dynamicity is with the variation of QoS requirement; if the latency metric is relaxed to 2000ms for *fd*, a larger input range is satisfied by the *fd*, the non-peak fusion with *ir* can now satisfy the QoS requirement for input up to 1000KB for the Cloud device. In summary, the fusion criterion can vary based on both the resource heterogeneity and QoS requirements.

B. Observations

From the above experiments, we make the following observations. Fusing functions with peak-resource assignments

is more expensive compared to fusion based on application characteristics, where fusion groups are assigned lower resource specifications. Application-specific characteristics can vary; in our experiments, input size, the number of inputs, etc., are varying attributes that impact the execution times. There is a need to consider application characteristics and QoS requirements to arrive at function-sizing decisions for cost-efficient executions. The fusion criterion can vary based on the resource type (Edge/Cloud) and QoS requirements. This necessitates techniques that provide fusion decisions dynamically based on QoS requirements and resource types. The proposed “RightFusion” platform addresses these gaps, the major components of the platform are discussed in Section IV.

III. RELATED WORK

Existing literature has explored function fusion for minimizing application end-to-end latency and reducing application execution costs. Authors in [18] present function fusion, bundling, pre-warming and resource allocation techniques for serverless Directed Acyclic Graphs (DAGs) in the Cloud using performance models with objectives to minimize latency and cost. Functions are fused if the intermediate data between two functions is high, and parallel invocations of the same function are bundled to reduce execution skew. Our work prioritizes fusion if two in-series functions have equivalent resource specifications and considers resource heterogeneity. XFaaS [19] presents a function placement and fusion technique for a multi-cloud FaaS provider scenario involving AWS and Azure. Their placement technique outputs sub-DAGs that can be placed either in a single Cloud Service Provider (CSP) or partitioned to different CSPs based on user-specified data-regulatory constraints and a goal to minimize end-to-end latency and cost. This work is limited by the fixed resource sizes of functions. Our work proposes a variable fusion criteria based on the application characteristics; the function fusion and right sizing are based on the incoming input and QoS metric in a heterogeneous resource setting. However, the aspect of data regulations with regard to function pinning to a layer is complementary to our proposed fusion algorithm. Authors in Costless [9] present an algorithm for function fusion and placement; they consider an IoT-Cloud scenario where an Image-processing pipeline is scheduled on Edge (AWS Greengrass) and Cloud (AWS Lambda); the objective is to reduce cost while being under a latency threshold using a Constrained Shortest Path algorithm. Fusionize [10] proposes fusing in-series functions to reduce billing from FaaS providers and to improve end-to-end application latency. Synchronous functions are fused to avoid double billing and asynchronous calls to functions are kept as a separate fusion group. Both Costless and Fusionize assume a single input size and a single resource configuration; our solution explores rightsizing FaaS instances based on varying application characteristics and QoS requirements. In the area of serverless resource management, FaaS Deliver [20] presents a solution to perform resource allocation and function placement in a Heterogeneous Computing Continuum. They present a Function Deliver Policy,

Algorithm 1 Retrieve Fusion Groups for incoming request

```

1: procedure RETRIEVERIGHTFUSIONGROUPS(app, input_params)
2:   app_dag, qos, uc  $\leftarrow$  RETRIEVEAPPINFO(app)
    $\triangleright$  Call The Right Fusion Algorithm
3:   (fg_map, res_spec)  $\leftarrow$ 
     RFALGORITHM(app_dag, qos, uc)
    $\triangleright$  Send the exec request to the first level fusion groups
4:   init_fg  $\leftarrow$  fg_map[l1]
5:   for fusion_group  $\in$  init_fg do
6:     exec_layer  $\leftarrow$  fusion_group
      $\triangleright$  Send Execution request to Cloud/Edge controller
7:     EXECFUSIONGROUP(fg_map, res_spec, l1, input_params)
8:   end for
9: end procedure
10: procedure RFALGORITHM(app_dag, qos, uc)
11:   fg_list  $\leftarrow$  MODIFIEDDFS(app_dag)
    $\triangleright$  Retrieve resource specification
12:   res_spec  $\leftarrow$ 
     ESTIMATERESOURCE(input_params, app_dag, qos, uc)
    $\triangleright$  Perform RightFusion based on res spec & fusion lists
13:   fg_map  $\leftarrow$  RIGHTFUSION(fg_list, res_spec)
14:   return (fg_map, res_spec)
15: end procedure

```

Algorithm 2 Resource Recommender

```

1: procedure ESTIMATERESOURCE(input_params, app_dag, qos, uc)
2:   for fi  $\in$  app_dag do
3:     for res_type  $\in$  all_res_types do
4:        $\triangleright$  Resource type can be Edge/Cloud
5:       for res_spec  $\in$  (min_res, max_res) do
6:         res_spec_map[fi][res_type][res_spec]  $\leftarrow$ 
           PREDICTEXECTIME(fi, input_params, res_type, res_spec)
7:       end for
8:     end for
9:   res_spec_map.APPLY_QOS_FILTER(qos)
10:  res_spec_map.APPLY_UC_FILTER(uc)
11:  return res_spec_map
12: end procedure

```

functions are returned (line 14). Once the fusion groups are obtained, requests to execute them are sent to the appropriate layer (Edge/Cloud) with the fusion group information, resource specification, and input parameters (lines 4 - 8). Finally, the functions are executed on a suitable worker node with their right-sized resources.

C. Resource Recommender

The resource recommender provides resource assignments to functions across both the Edge and Cloud layers, subject to the application's input parameters, user-specified QoS requirements, and constraints. We collect the execution trace of each function in an application workflow for different input sizes and different resource specifications across the Edge and Cloud layers. A Machine Learning (ML) model is trained per function on the collected execution trace, which predicts the execution time (et_{f_i}) for a function f_i , given an execution layer (Edge/Cloud), resource specification (CPU and memory), and incoming input parameters (i_1 to i_n) as shown below:

$$\langle E|C, cpu, mem, i_1, i_2, \dots, i_n, et_{f_i} \rangle$$

The Algorithm. 2 highlights the steps involved in providing resource recommendation. For each function in the application DAG (line 2), the execution times are calculated for each

resource type (line 3). The recommender predicts the execution time for various resource specifications from minimum to maximum (user-specified peak resource) and records the execution times (lines 4 and 5). Once the resource specifications are recorded, the entries where the execution times exceed the QoS metric are filtered out (line 9). Further, entries which override user constraints are also filtered out (line 10). Finally, resource specifications for the functions are returned (line 11).

For the application scenarios in this work in Section V, to predict execution times, we have used simplistic ML models to illustrate the benefits of RightFusion. ML models were used to capture the non-trivial correlations between application characteristics and execution times. For the Text Inferencing workflow, DecisionTree Regressor was used, which gave a Mean Absolute Percentage Error (MAPE) of 4.8%, 5.7%, and 2.97% for *sd*, *pd*, and *ts* respectively. For the Face Detection workflow, KNN Regressor was used, which gave a MAPE of 11.2%, 10.8%, and 8.7% for *fd*, *ir*, and *fc* respectively. For the NSFW Detection workflow, we used DecisionTree Regressor, which gave a MAPE of 7.5%, 6.8%, 10.8%, and 4.2% respectively for *nsfw*, *anno*, *multi ir*, and *image concat* respectively. The selection were based on the Ares [26] tool. In practical serverless deployments, 80% of the applications have less than 5 functions [27]. The number of resource types are between 2 to 5 [28] in commercial serverless offerings. The proposed technique is feasible in these scenarios.

V. EVALUATION USING APPLICATION SCENARIO

We consider three representative inferencing applications in our evaluation: NSFW Detection workflow, Face detection Workflow, and Text Inferencing workflow.

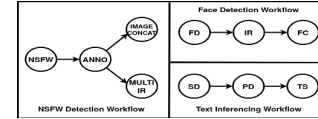


Fig. 4: Application Scenario

Text Inferencing Workflow: This workflow consists of 3 functions spam detection (*sd*), profanity detection (*pd*), and text summarization (*ts*) as seen in Fig. 4. Spam detection uses a Naive Bayes' probabilistic model for classifying text as spam; Profanity Detection uses 'alt-profanity-checker' Python module [29], Text Summarization is based on Python's NLTK library [30]. All three functions are single-threaded, the application characteristic used is the document size.

Face Detection Workflow: Face Detection workflow consists of 3 functions, namely, face detection (*fd*), image resizing (*ir*), and format conversion (*fc*) as seen in Fig. 4. Face Detection uses HaarCascades classifier to detect faces using OpenCV module. Image Resizing resizes the images using the 'python-image-resize' module. Format conversion converts the image format from JPEG to webp using Python 'Pillow' module. All the functions except *fd* are single-threaded, and *fd* is a multi-threaded function. The application characteristic used is the image size and the resolution of the image.

NSFW Detection Workflow: This workflow consists of 4 functions namely *nsfw*, *annotation*, *multi image resizing*, and *image concatenation* as seen in Fig. 4. The *nsfw* function uses a PyTorch library and ResNet50 network [16] for classifying images. The *annotation* function uses OpenCV library to annotate the image with the watermark of whether image is NSFW or not. *image concatenation* uses OpenCV to concatenate all the images coming from annotation. *multi image resizing* creates resized images of the incoming images from annotation using Python Pillow library. All the functions except *nsfw* are single threaded, *nsfw* is a multi-threaded function. The application characteristic used are image size, the resolution of the image, number of images, and total size of all images.

VI. RESULTS

The evaluation highlights the benefits of the proposed technique “RightFusion” against two baselines, “B1-NoFusion” and “B2-PeakFusion”, in terms of performance and cost. The B1-NoFusion represents the executions in the existing serverless platforms [11], [12] [13], [14] where the functions in an application workflow are executed one after another without fusing any functions. B2-PeakFusion forms fusion groups of all linear sub-paths (excluding branches) in a workflow.

TABLE I: Application, QoS metrics & Resource Assignment

Text Inf. Pipeline	QoS (relaxed)	QoS (strict)	Peak Resource
<i>spam</i>	100ms	50ms	(1vCPU, 1GB)/C
<i>profanity</i>	100ms	50ms	(1vCPU, 1GB)/C
<i>text summarization</i>	250ms	200ms	(1vCPU, 1GB)/C
Face Det. Pipeline	QoS (relaxed)	QoS (strict)	Peak Resource
<i>face detection</i>	1750ms	1500ms	(2vCPU, 2GB)/C
<i>image resizing</i>	1000ms	750ms	(1vCPU, 1GB)/C
<i>format conversion</i>	3000ms	2750ms	(1vCPU, 1GB)/C
NSFW Pipeline	QoS (relaxed)	QoS (strict)	Peak Resource
<i>nsfw</i>	5000ms	800ms	(2vCPU, 2GB)/C
<i>annotation</i>	3500ms	3000ms	(1vCPU, 1GB)/C
<i>image concat</i>	7000ms	6000ms	(1vCPU, 1GB)/C
<i>multi image resize</i>	4000ms	3000ms	(1vCPU, 1GB)/C

A. Experimental Setup

The Serverless controller, Cloud controller, Resource Manager, Cloud Data service, Cloud Fusion Worker, and the FaaS functions (Docker containers) are deployed on Intel(R) Xeon(R) Silver 4310 CPU @ 2.10GHz node with 12 physical cores/socket with two sockets, 128GB DDR4 RAM. The Edge controller, event manager, and data services are hosted on an Intel(R) Core(TM) i7-8700 CPU @ 3.20GHz with 12 logical cores (6 physical cores), 16GB DDR4 RAM. The Edge Fusion Worker is deployed on Raspberry Pi4 Model B quad-core Cortex-A72 (ARM v8) 64-bit SoC @ 1.5GHz, 4GB RAM.

Workload: The workload involves 250 requests to each technique namely RF-Strict, B1-NoFusion, B2-PeakFusion, and RF-Relaxed. This results in 1000 function executions for NSFW pipeline (250 x 4 functions), and 750 for Text Inferencing and Face Detection pipeline (250 x 3 functions).

Text Inferencing: For the text inferencing workflow, we vary the text document sizes from 2KB to 100KB, the input text size is normally distributed with the mean $\mu = 35$ KB,

$\sigma = 25$ KB. The text documents include news articles [31], Wikipedia pages retrieved from the Internet.

NSFW Workflow: For the NSFW pipeline, as image sizes can vary considerably [17] [32] in the Web Pages from tens of KBs to MBs. To represent this wide range, we use images of different sizes from 50KB to 2MB, with different number of images from 1 through 5. The image size is normally distributed with $\mu = 500$ KB and $\sigma = 400$ KB, the number of images are sampled randomly between 1 and 5.

Face Detection Workflow: For this workflow, the workload is similar to the NSFW’s workload except that only one image per request is sent instead of multiple.

Setup: In B1-NoFusion, functions are assigned with peak resource assignment based on resource specification in Table. I (column 4). For the baseline B2-PeakFusion, functions *nsfw* and *annotation* are fused in the NFSW detection pipeline; the fusion group gets (2vCPU, 2GB)-Cloud, as the resource assignment (since it is the maximum between *nsfw* and *annotation*). Similarly, in the case of text inferencing, the peak fusion involves *sd*, *pd*, and *ts* all in a single fusion group with (1vCPU,1GB)-Cloud being the resource assignment. For the Face Detection pipeline, there is a single fusion group involving *fd*, *ir*, and *fc*, with (2vCPU, 2GB)-Cloud as the resource assignment. All the functions are pre-warmed before running the experiments. The application workflows typically constitute back-end services that are invoked based on user-activity, with access to user activity pattern and keep-alive policy of serverless provider, the functions can be kept-warm.

B. Performance

The goal of the experiment is to compare the effect of function fusion on the aspect of performance among B1-NoFusion, B2-PeakFusion, and RightFusion for the QoS strict and relaxed requirements of Table. I. The end-to-end latency is the performance metric in our evaluation; it depends on three factors, namely, function execution layer (Edge/Cloud), resource assignment (peak/non-peak right-sized resource), and the number of remote transfers (access to data-store through *get()*/*put()*). For 3 applications, Text Inferencing, Face Detec-

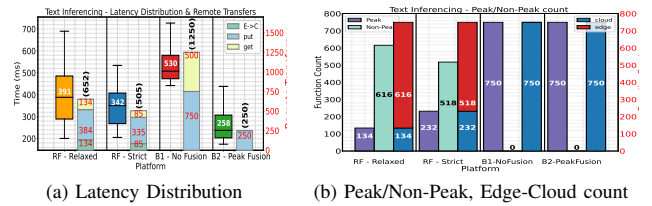


Fig. 5: Text Inferencing: Latency and Execution count split

tion, and NSFW Detection, the plots in the Fig. 5a, 6a, and 7a indicate the average end-to-end latency for different techniques B1-NoFusion, B2-PeakFusion, RightFusion for QoS strict requirement (RF-Strict), and RightFusion for QoS relaxed requirement (RF-Relaxed). The remainder of the section explains the performance on QoS strict and QoS relaxed.

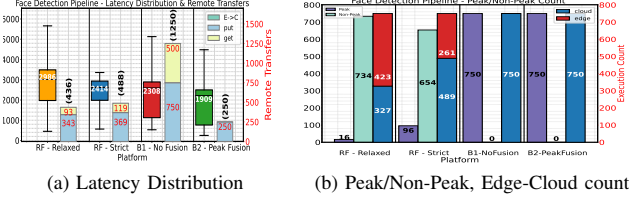


Fig. 6: Face Detection: Latency and Execution count split

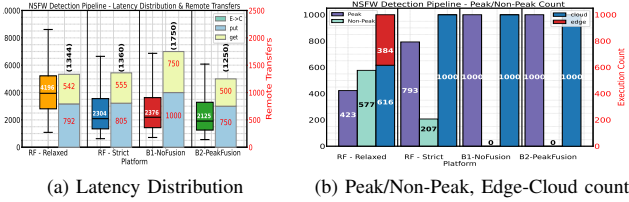


Fig. 7: NSFW Detection: Latency and Execution count split

QoS Strict Requirement: All three techniques, B1-NoFusion, B2-PeakFusion, and RF-Strict, satisfy the strict QoS requirements. B2-PeakFusion takes the lowest time across all applications as it executes all the functions with peak resources on Cloud, it avoids external co-ordination for fused functions from the serverless controller, and has the lowest data transfers as observed in the Y2-axis of plots in the Fig. 5a, 6a, and 7a. In Text Inferencing, RF-Strict outperforms B1-NoFusion even though the number of non-peak execution counts is higher compared to B1-NoFusion as seen in the Y1-axis in plots of Fig. 5b. The low number of remote transfers (505) in RF-Strict compared to B1-NoFusion (1250) compensate for the non-peak executions. In NSFW detection, RF-Strict is marginally better than B1-NoFusion, here non-peak executions are performed on Cloud as seen in the Y1-axis of plots Fig. 7b; the reduced remote transfers compensate for the non-peak executions at Cloud. In Face Detection, even though RF-Strict has a low amount of remote transfers (488) compared to B1-NoFusion (1250), the non-peak execution on Edge devices as seen in Y1 and Y2 axis of plots of Fig. 6b increase the average end-to-end latency. However, both the techniques meet the user-specified QoS requirements.

QoS relaxed requirement: In order to examine the impact of QoS requirement on function deployment across different resource layers, we relax the latency metric for all the 3 applications as indicated in the 2nd column of Table. I. The relaxed QoS requirement enable the RightFusion technique to deploy more non-peak functions for all the applications (indicated by the RF-Relaxed label) as seen in the non-peak execution count in Y1-axis of plots in Fig. 5b, Fig. 6b, and Fig. 7b. For NSFW, with RF-Relaxed, 384 functions are now executed with non-peak resources on Edge layer, as observed with Edge-count in Y2-axis of plots of Fig. 7b. This demonstrates the ability of the platform to “right size” resources in a heterogeneous resource pool based on the user-specified QoS requirements. In

summary, the RightFusion technique prioritizes the selection of lower capacity resources for execution and fusion, i.e., choosing the Edge or a “non-peak” Cloud resource as long as user-specified QoS requirements are satisfied.

C. Cost

This section discusses the cumulative cost (refer Eqn. 1) of function executions for different techniques. The workload in section VI-A is used for the cost analysis. The cost of function execution on the Edge is considered as 0 based on the cost modeling of [9]; on Cloud, it is resource size (vCPU count) multiplied by execution time (in ms), expressed as usage in vCPU ms. PeakFusion has the highest resource usage in all

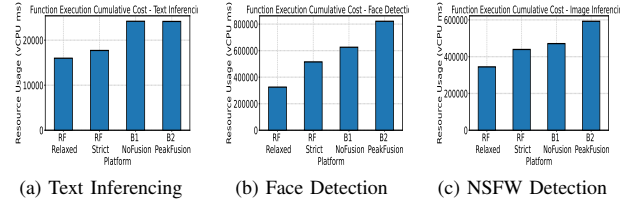


Fig. 8: Cumulative Function Execution Cost for applications

applications, it is treated as 100%. For text inferencing, RF-Strict performs 518 non-peak executions out of 750; all the 518 non-peak executions are on the Edge layer as observed in the Y2-axis of Fig. 5b. The non-peak executions result in significant cost savings as highlighted in Fig. 8a; the RF-Strict uses 35% lower resources as compared to both B1-NoFusion and B2-PeakFusion while still meeting the QoS requirements. Similarly, in the case of the Face Detection pipeline, for the RF-Strict technique, out of 654 non-peak executions, 261 execute on Edge as observed in plots in the Y2-axis of Fig. 6b, resulting in 37% lower resource usage compared to B2-PeakFusion and 18% lower usage compared to B1-NoFusion as seen in plots of Fig. 8b. Finally, in the case of the NSFW detection pipeline, though the RF-strict technique executes all the functions on the Cloud layer as seen in Fig. 7b, due to the resource right-sizing, 207 out of 1000 total function executions are non-peak executions. RF-Strict uses 25% lower resources compared to B2-PeakFusion, 11% lower compared to B1-NoFusion, as highlighted in the plots in Fig. 8c. With QoS-relaxed requirements, each application uses the lowest resources compared to RF-Strict, B1-NoFusion, and B2-PeakFusion, as it performs the most number of non-peak executions. In summary, the right-sizing prioritizes the selection of least-cost resources for function executions; it prioritizes the choice of the Edge layer and only chooses the Cloud layer when the Edge layer cannot meet the QoS requirements. In the Cloud layer, RightFusion also tries to perform non-peak resource assignment to functions, thereby using lower resources to meet the QoS requirements.

D. Varying QoS requirements and Function Fusion

RightFusion uses QoS requirements to decide the resource layer of function fusion. For QoS-strict requirement, the fusion

groups are often deployed in the Cloud, for QoS-relaxed, Edge layer is chosen based on performance. Plots in Fig. 9 highlight

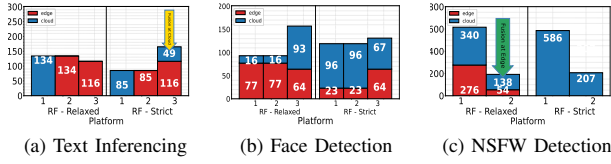


Fig. 9: Function Fusion counts

function fusion at different layers for all the 3 applications, the X-axis indicate the length of fusion group, Y-axis indicates the fusion count. For text inferencing, when the QoS requirements are strict, the fusion groups start shifting towards Cloud as seen in Fig. 9a (yellow arrow). Similarly, for the NSFW Detection, the fusions that were performed at Cloud for RF-Strict start shifting towards Edge (green arrow) in Fig. 9c when the QoS requirements are relaxed, demonstrating the dynamicity of Function Fusion based on user-specified QoS requirements.

VII. CONCLUSION AND FUTURE WORK

Existing works on function fusion are limited by peak resource assignments, leading to high execution costs. This paper motivates using varying application characteristics, QoS requirements, and right-sized resources in a heterogeneous resource setting for cost-efficient function fusion through RightFusion. RightFusion uses ML techniques to recommend low-cost resource assignments, and the RightFusion algorithm forms fusion groups on the Edge and Cloud layer based on user constraints and QoS requirements. Experimental evaluation of 3 different real-world applications shows significant resource savings above 35% for the proposed algorithm compared to PeakFusion. In future, we plan to support other QoS attributes like privacy, security requirements and generalize the platform to support different on-premise/commercial FaaS deployments.

REFERENCES

- [1] "Aws lambda pricing," <https://aws.amazon.com/lambda/pricing/>, 2024.
- [2] A. Palade, Kazmi *et al.*, "An evaluation of open source serverless computing frameworks support at the edge," in *2019 IEEE World Congress on Services (SERVICES)*, vol. 2642-939X, 2019, pp. 206–211.
- [3] M. S. Aslanpour, Toosi *et al.*, "Serverless edge computing: Vision and challenges," in *Proceedings of the 2021 Australasian Computer Science Week Multiconference*, ser. ACSW '21. New York, NY, USA: Association for Computing Machinery, 2021. [Online]. Available: <https://doi.org/10.1145/3437378.3444367>
- [4] S. K. R and J. Lakshmi, "Hybrid serverless platform for service function chains," in *2023 IEEE 16th International Conference on Cloud Computing (CLOUD)*, 2023, pp. 493–504.
- [5] A. Singhvi, Banerjee *et al.*, "Granular computing and network intensive applications: Friends or foes?" in *Proceedings of the 16th ACM Workshop on Hot Topics in Networks*, ser. HotNets-XVI. New York, NY, USA: Association for Computing Machinery, 2017, p. 157–163. [Online]. Available: <https://doi.org/10.1145/3152434.3152450>
- [6] M. Savi and a. o. Banfi, "Serverless computing for nfv: Is it worth it? a performance comparison analysis," in *2022 IEEE International Conference on Pervasive Computing and Communications Workshops and other Affiliated Events (PerCom Workshops)*, 2022, pp. 680–685.
- [7] W. Lloyd *et al.*, "Serverless computing: An investigation of factors influencing microservice performance," in *2018 IEEE International Conference on Cloud Engineering (IC2E)*, 2018, pp. 159–169.
- [8] A. Mahgoub *et al.*, "ORION and the three rights: Sizing, bundling, and prewarming for serverless DAGs," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. Carlsbad, CA: USENIX Association, Jul. 2022, pp. 303–320. [Online]. Available: <https://www.usenix.org/conference/osdi22/presentation/mahgoub>
- [9] T. Elgamal *et al.*, "Costless: Optimizing cost of serverless computing through function fusion and placement," in *2018 IEEE/ACM Symposium on Edge Computing (SEC)*, 2018, pp. 300–312.
- [10] T. Schirmer, J. Scheuner, T. Pfandzelter, and D. Bermbach, "Fusionize: Improving serverless application performance through feedback-driven function fusion," in *2022 IEEE International Conference on Cloud Engineering (IC2E)*, 2022, pp. 85–95.
- [11] S. Hendrickson, Sturdevant *et al.*, "Serverless computation with open-lambda," in *8th {USENIX} Workshop on Hot Topics in Cloud Computing (HotCloud 16)*, 2016.
- [12] "Apache openwhisk," github.com/apache/openwhisk, year = 2024.
- [13] "Aws lambda," <https://aws.amazon.com/lambda/>, 2024.
- [14] "Azure functions," azure.microsoft.com/en-in/products/functions, 2024.
- [15] S. K. R and J. Lakshmi, "Qos aware faas platform," in *2021 IEEE/ACM 21st International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, 2021, pp. 812–819.
- [16] emilantolo, "pytorch model for nsfw detection," https://github.com/emilantolo/pytorch_nsfw_model, 2024.
- [17] A. Tahir *et al.*, "Deconstructing google's web light service," in *Proceedings of The Web Conference 2020*, 2020, pp. 884–893.
- [18] A. Mahgoub, Yi *et al.*, "Wisefuse: Workload characterization and dag transformation for serverless workflows," *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 6, no. 2, jun 2022. [Online]. Available: <https://doi.org/10.1145/3530892>
- [19] A. Khochare, T. Khare, V. Kulkarni, and Y. Simmhan, "Xfaas: Cross-platform orchestration of faas workflows on hybrid clouds," in *2023 IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, 2023, pp. 498–512.
- [20] G. Yu, P. Chen *et al.*, "Faasdeliver: Cost-efficient and qos-aware function delivery in computing continuum," *IEEE Transactions on Services Computing*, vol. 16, no. 5, pp. 3332–3347, 2023.
- [21] K. R. Sheshadri and J. Lakshmi, "Qos aware faas for heterogeneous edge-cloud continuum," in *2022 IEEE 15th International Conference on Cloud Computing (CLOUD)*, 2022, pp. 70–80.
- [22] Z. Zhou, Y. Zhang, and C. Delimitrou, "Aquatope: Qos-and-uncertainty-aware resource management for multi-stage serverless workflows," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ser. ASPLOS 2023. New York, NY, USA: Association for Computing Machinery, 2022, p. 1–14. [Online]. Available: <https://doi.org/10.1145/3567955.3567960>
- [23] A. Raza, Akhtar *et al.*, "Configuration and placement of serverless applications using statistical learning," *IEEE Transactions on Network and Service Management*, vol. 20, no. 2, pp. 1065–1077, 2023.
- [24] R. Patsch and K. M. Göschka, "Optimizing the cost-performance ratio of faas deployments," in *European Conference on Service-Oriented and Cloud Computing*. Springer, 2023, pp. 263–275.
- [25] V. M. Bhasi *et al.*, "Cypress: input size-sensitive container provisioning and request scheduling for serverless platforms," in *Proceedings of the 13th Symposium on Cloud Computing*, ser. SoCC '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 257–272. [Online]. Available: <https://doi.org/10.1145/3542929.3563464>
- [26] J. Elms *et al.*, "Ares: An automl regression service for data analytics and novel data-centric visualizations."
- [27] S. Eismann *et al.*, "The state of serverless applications: Collection, characterization, and community consensus," *IEEE Transactions on Software Engineering*, vol. 48, no. 10, pp. 4152–4166, 2022.
- [28] D. Kelly, F. Glavin, and E. Barrett, "Serverless computing: Behind the scenes of major platforms," in *2020 IEEE 13th International Conference on Cloud Computing (CLOUD)*, 2020, pp. 304–312.
- [29] dimitrymi and menelaoskotoglou, "alt-profanity-check," <https://pypi.org/project/alt-profanity-check/>, 2024.
- [30] U. Malik, "text summarization with nltk in python," <https://stackabuse.com/text-summarization-with-nltk-in-python/>, 2024.
- [31] R. Mishra, "news category dataset," <https://www.kaggle.com/datasets/rmisra/news-category-dataset>, 2024.
- [32] P. Casas, Wassermann *et al.*, "Not all web pages are born the same content tailored learning for web qoe inference," in *2022 IEEE International Symposium on Measurements & Networking (M&N)*, 2022, pp. 1–6.